

Requirement 4 — Slightly Non-Stationary Environment

Note di studio – Online Learning Applications, Prof. Castiglioni

Come usare queste note. Non sono lo script della presentazione (per quello vedi l'artifact web) – sono pensate per essere lette e capite a fondo, incluse le domande che il professore potrebbe fare. Ogni sezione con immagine cita il file esatto nella cartella `outputs/r4/`.

Contents

1	Inquadramento generale	1
2	L'ambiente: shock piecewise-stazionari	1
3	Le tre strategie	2
3.1	Sliding-Window Combinatorial-UCB	2
3.2	CUSUM Combinatorial-UCB	2
3.3	Primal-Dual (Req3, riusato)	2
4	Metodologia dei benchmark	3
5	Risultati principali	3
6	È un problema di taratura?	3
7	Perché è strutturale: la teoria in breve	4
8	Domande d'esame plausibili	5
9	Numeri chiave da ricordare	6

1 Inquadramento generale

Requirement 4 riusa *esattamente* lo stesso problema di Requirement 2 e 3: $N = 4$ campagne, budget condiviso $B = 1600$, orizzonte $T = 10\,000$, grafo dei conflitti $\{(0,1), (2,3)\}$, valori $v = [0.8, 0.8, 0.9, 0.9]$, griglia di bid `linspace(0,1,11)` e feedback completo (osserviamo m_t , la miglior offerta avversaria, alla fine di ogni round).

L'unica cosa che cambia rispetto a Req3 è il **tipo di non-stazionarietà**: Req3 usa `mode='drift'` (cambio continuo, ogni round), Req4 usa `mode='shocks'` (cambio discreto, a blocchi). È la stessa classe di ambiente (`AdversarialMultiCampaignEnv`), solo riparametrizzata.

2 L'ambiente: shock piecewise-stazionari

L'orizzonte $T = 10\,000$ è diviso in 5 blocchi da 2000 round (`block_size=2000, n_regimes=5`). Prima dell'inizio della simulazione vengono estratti 5 regimi $\text{Beta}(\alpha_i, \beta_i)$ con media $\mu_i \sim$

	Req3 – drift	Req4 – shocks
Tipo di cambiamento	continuo, ogni round	discreto, a blocchi
Meccanismo	$m_t \sim \text{Beta}(\alpha_t, \beta_t)$, media sinusoidale	5 blocchi da 2000 round, regime i.i.d. per blocco
Prevedibilità	la direzione del cambiamento è liscia	nessun preavviso al confine di blocco
Benchmark naturale	OPT^A (miglior fissa nel tempo)	clairvoyant piecewise (per-blocco)

Table 1: Confronto tra i due tipi di non-stazionarietà usati in Req3 e Req4.

Uniform(0.1, 0.9); a ogni blocco viene assegnato uno di questi 5 regimi **scelto uniformemente a caso** (quindi un regime può ripetersi, non è garantito un giro completo su tutti e 5).

Dentro un blocco, m_t è i.i.d. dal regime assegnato. Al confine tra blocchi, la distribuzione cambia istantaneamente e senza segnale. L'agente non conosce:

- dove sono i confini dei blocchi;
- quanti regimi distinti esistono;
- se un regime già visto si ripeterà.

Limite noto, da citare se chiesto: poiché i regimi sono riassegnati ai blocchi in modo i.i.d., non c'è garanzia che i 5 blocchi coprano tutti i 5 regimi distinti – in alcune run un regime può comparire due volte e un altro zero volte. Non invalida i risultati (il benchmark piecewise usa comunque le distribuzioni vere di ciascun blocco), ma è un dettaglio implementativo onesto da conoscere se il professore lo chiede.

3 Le tre strategie

3.1 Sliding-Window Combinatorial-UCB

Estende il `CombinatorialUCBAgent` di Req2 mantenendo le statistiche solo sugli ultimi W round (finestra temporale, non per-braccio). Con $W = 500$ (un quarto della lunghezza di un blocco): abbastanza corto da smaltire i dati stantii nei primi ~ 500 round dopo uno shock, abbastanza lungo da lasciare ~ 1500 round di sfruttamento stabile.

3.2 CUSUM Combinatorial-UCB

Stessa base, ma con un rilevatore di cambiamento Page (1954) CUSUM per ogni cella (campagna, bid): quando la statistica CUSUM supera una soglia h , la cella viene resettata (trattata come mai esplorata). Parametri usati: $h = 8.0$, $\varepsilon = 0.02$, $U_T = 4$ (numero atteso di cambi di regime). Con probabilità $\alpha = \sqrt{U_T \log(T/U_T)/T}$ gioca un'azione casuale come rete di sicurezza contro derive lente che CUSUM potrebbe non cogliere subito.

3.3 Primal-Dual (Req3, riusato)

È *lo stesso identico agente* di Requirement 3 (Hedge per campagna + discesa duale OGD su λ), **nessuna modifica strutturale** – solo i tassi di apprendimento sono stati ritirati per Req4: `hedge_eta=0.16`, `ogd_eta=0.003`, con `budget_pacing=True` (target duale adattivo $\rho_t = B_{\text{rimanente}}/T_{\text{rimanente}}$).

D: perché Primal-Dual non è stato ridisegnato per Req4?

Perché il punto pedagogico del requirement è proprio verificare cosa succede quando si *riusa senza modifiche strutturali* un algoritmo con garanzia di best-of-both-worlds in un ambiente per cui quella garanzia non è stata pensata. Ridisegnarlo avrebbe nascosto il fenomeno interessante.

4 Metodologia dei benchmark

Come in Req3, un clairvoyant dinamico (che conosce ogni m_t realizzato) gonfia il regret di un termine lineare in T per la disuguaglianza di Jensen – nessun learner può avere regret sub-lineare contro di esso in un’asta first-price. Usiamo quindi tre benchmark, in ordine di importanza:

Livello	Cosa conosce	Funzione
Primario	confini dei blocchi + distribuzione vera per blocco (non i singoli m_t)	<code>compute_piecewise_expected_clairvoyant</code>
Secondario (OPT ^A)	miglior distribuzione fissa nel tempo (continuità con Req3)	<code>compute_clairvoyant_multi</code>
Riferimento (prophet)	ogni m_t realizzato	<code>compute_clairvoyant_dynamic_multi</code>

Table 2: Gerarchia dei tre benchmark usati in Req4.

Il clairvoyant piecewise è il target naturale per SW-UCB e CUSUM-UCB: ha esattamente le garanzie teoriche di letteratura (es. Garivier & Moulines, 2011) contro cui questi algoritmi sono progettati.

5 Risultati principali

Agente	Piecewise	OPT ^A	Prophet	Costo finale / B
Primal-Dual (Req4-tuned)	1426.1	626.8	2699.7	1456.4 / 1600 (91.0%)
CUSUM Combinatorial-UCB	1430.0	630.7	2703.6	1599.3 / 1600 (99.96%)
Sliding-Window Comb.-UCB	1947.6	1148.3	3221.2	1599.2 / 1600 (99.95%)

Table 3: Risultati finali medi su 20 trial, $T = 10\,000$ (numeri dal notebook eseguito).

Lettura critica. Il ranking sul regret non racconta tutta la storia. Guardando la spesa di budget, Primal-Dual è chiaramente diverso dagli altri due: CUSUM e Sliding-Window hanno un meccanismo esplicito di reset innescato da un cambio di regime rilevato; Primal-Dual no. Questo è il punto di partenza della Sezione 6.

6 È un problema di taratura?

Per verificare se il sotto-utilizzo del budget da parte di Primal-Dual fosse dovuto a una taratura non ottimale (ereditata da Req3, pensata per il drift continuo), abbiamo confrontato **lo stesso agente, sullo stesso ambiente shocks**, con due configurazioni di learning rate:

Conclusione empirica. Il retuning *aiuta*: il regret scende da 1777 a 1426 (arriva a pareggiare CUSUM) e la spesa sale dal 75% al 91%. Ma **nessuna delle due configurazioni si avvicina al 99.9% di CUSUM e Sliding-Window**. Aumentare il tasso di apprendimento del primale (Hedge) accelera l’adattamento *dentro* ogni blocco; non risolve il ritardo con cui λ reagisce *subito dopo* uno shock – quel ritardo è strutturale al meccanismo di discesa duale OGD, che non riceve mai un segnale esplicito "il regime è appena cambiato".

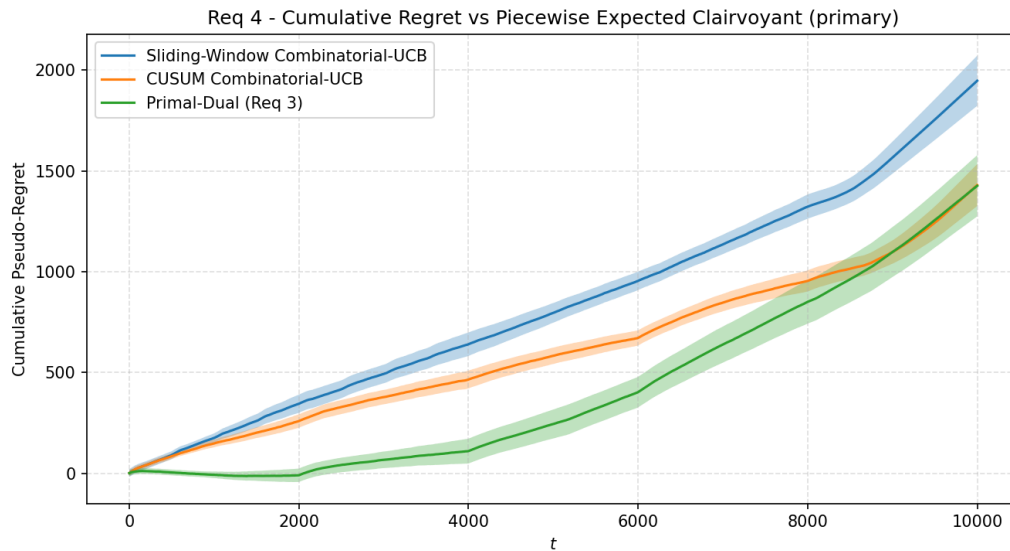


Figure 1: Regret cumulativo contro il clairvoyant piecewise (benchmark primario). Primal-Dual e CUSUM sono sostanzialmente appaiati; Sliding-Window è nettamente peggiore.

Configurazione	Piecewise	OPT ^A	Prophet	Costo / B	λ fin.
Req3 eta (baseline, non ritarato)	1777.4	978.0	3051.0	1193.9/1600 (74.6%)	0.575
Req4 tuned (hedge=0.16, ogd=0.003)	1426.1	626.8	2699.7	1456.4/1600 (91.0%)	1.018

Table 4: Confronto Primal-Dual: tassi di Req3 (non ritarati per lo shock discreto) vs tassi ritarati per Req4.

D: perché non avete cercato tassi ancora più aggressivi?

Un tasso di apprendimento di Hedge troppo alto causa un'instabilità numerica reale: i pesi moltiplicativi $w \leftarrow \exp(-\eta \cdot \text{loss})$ vanno in underflow a zero esatto, e la normalizzazione $w / \sum w$ diventa $0/0 = \text{NaN}$. C'è quindi un limite fisico oltre al quale l'algoritmo smette semplicemente di funzionare, non solo di migliorare.

7 Perché è strutturale: la teoria in breve

- **Hedge (il primale di Primal-Dual)** ha una garanzia di *regret statico/esterno*: converge, in aspettativa, alla miglior distribuzione *fissa nel tempo* – esattamente la definizione di OPT^A. Non ha alcun meccanismo di *dimenticanza* o di rilevamento di cambio regime incorporato nell'algoritmo.
- **OGD sul duale** λ insegue un target ρ_t con un passo di gradiente proporzionale allo scarto tra spesa realizzata e target – una risposta *graduale*, adatta a un target che si muove con continuità (Req3, drift), non a un target che salta discretamente (Req4, shocks).
- **SW-UCB e CUSUM-UCB**, al contrario, hanno un meccanismo di *tracking regret* esplicito: la finestra scorrevole scarta attivamente i dati vecchi; CUSUM resetta esplicitamente alla detection di uno shift. Sono progettati, in letteratura, proprio per ambienti piecewise-stazionari.

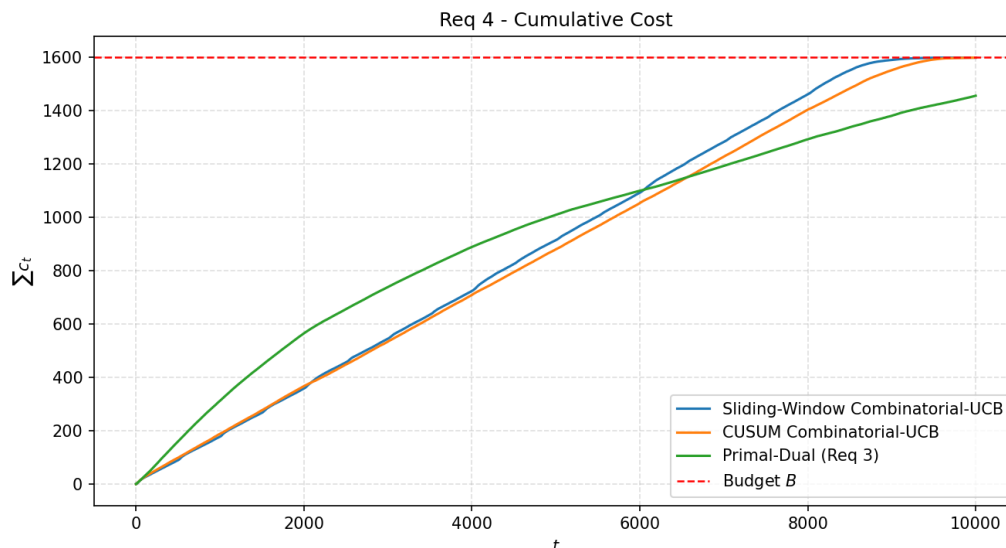


Figure 2: Costo cumulativo dei tre agenti contro la linea del budget. CUSUM e Sliding-Window esauriscono quasi tutto il budget; Primal-Dual si ferma al 91%.

La frase da avere pronta per l'esame: *“Lo stesso identico algoritmo Primal-Dual eccelle nel drift continuo di Req3 (dove raggiunge il 98% circa di spesa budget) e resta strutturalmente più conservativo negli shock discreti di Req4 (91% anche dopo retuning) – non per un errore di implementazione, ma perché la garanzia di Hedge è di regret statico contro la miglior distribuzione fissa, non di tracking contro un target che cambia bruscamente.”*

8 Domande d'esame plausibili

D: il professore si aspetta questo risultato "deludente" per Primal-Dual?

Molto probabilmente sì. Il testo del requirement chiede esplicitamente di riusare l'algoritmo di Req3 come *riferimento*, non di farlo vincere. Un risultato in cui Primal-Dual è sistematicamente più conservativo, con una spiegazione teorica precisa del *perché*, è pedagogicamente più interessante di un pareggio a tre su tutto – è la dimostrazione di aver capito il dominio di validità delle garanzie teoriche, non solo di saper implementare l'algoritmo.

D: com'è collegato l'ambiente di Req4 a quello di Req3?

Stessa classe `AdversarialMultiCampaignEnv`, stesso protocollo `round()`, stesso feedback completo su m_t . Cambia solo il metodo di generazione della sequenza: `_build_drift` (sinusoide continua per Req3) vs `_build_shocks` (blocchi piecewise per Req4). Tutti gli altri parametri del problema (N , B , T , valori, conflitti) sono identici e importati dallo stesso `req3_config.py`.

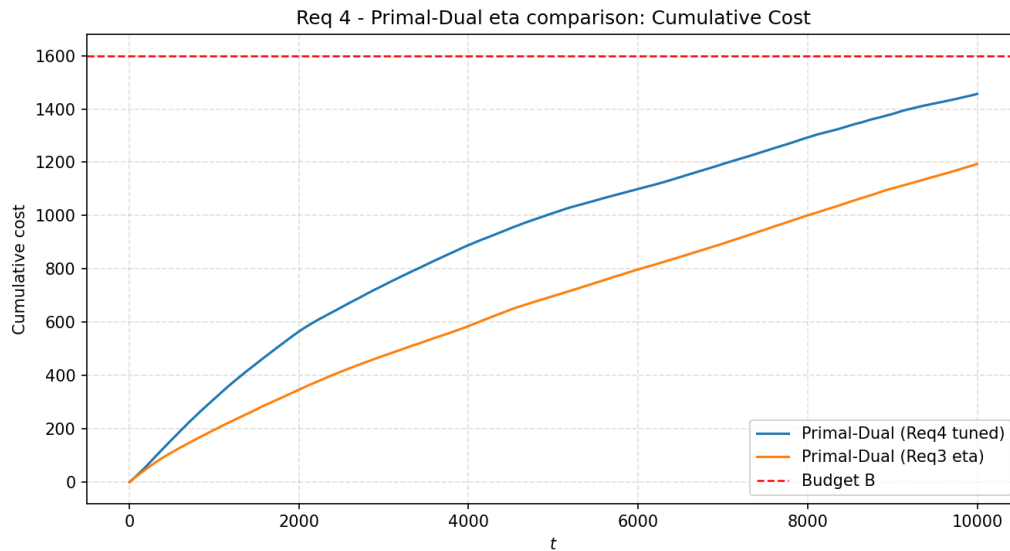


Figure 3: Costo cumulativo: Req3 eta (arancio, baseline) vs Req4 tuned (blu). Nessuna delle due curve tocca mai la linea rossa del budget.

D: perché Sliding-Window è l'agente peggiore, e non semplicemente "diverso"?

Con $W = 500$ e blocchi da 2000 round, SW-UCB butta via dati validi molto prima che il blocco finisca, restando perennemente in una fase di esplorazione rumorosa invece di sfruttare stabilmente. È un compromesso deliberato (finestra corta = adattamento rapido ma varianza alta durante lo sfruttamento); con W pari alla lunghezza del blocco l'agente adatterebbe più lentamente ma sfrutterebbe meglio – un secondo trade-off esplorabile, non mostrato nei risultati principali.

D: il fatto che i regimi possano ripetersi tra i blocchi è un bug?

No, è una scelta di design dell'ambiente (`_build_shocks` assegna i regimi ai blocchi in modo i.i.d., non a rotazione). È onesto segnalarlo come limite noto se chiesto, ma non invalida né i risultati né il benchmark piecewise, che usa comunque le distribuzioni vere – ripetute o no – di ciascun blocco specifico.

9 Numeri chiave da ricordare

- Ambiente: 5 blocchi $\times$ 2000 round, $T = 10\,000$, $B = 1600$, stessi parametri di Req2/Req3.
- Regret piecewise finale: **Primal-Dual 1426 $\approx$ CUSUM 1430 $\ll$ Sliding-Window 1948.**
- Budget speso: CUSUM e SW $\approx 99.9\%$, Primal-Dual **91.0%** (era 74.6% coi tassi di Req3, non ritirati).
- Retuning Primal-Dual: `hedge_eta` 0.015 $\rightarrow$ 0.16, `ogd_eta` 0.017 $\rightarrow$ 0.003.
- CUSUM: $h = 8.0$, $\varepsilon = 0.02$, media **145 reset** su 38 celle per trial.
- Messaggio centrale: il gap di Primal-Dual è **strutturale** (regret statico vs tracking), non risolvibile con il solo tuning dei learning rate.

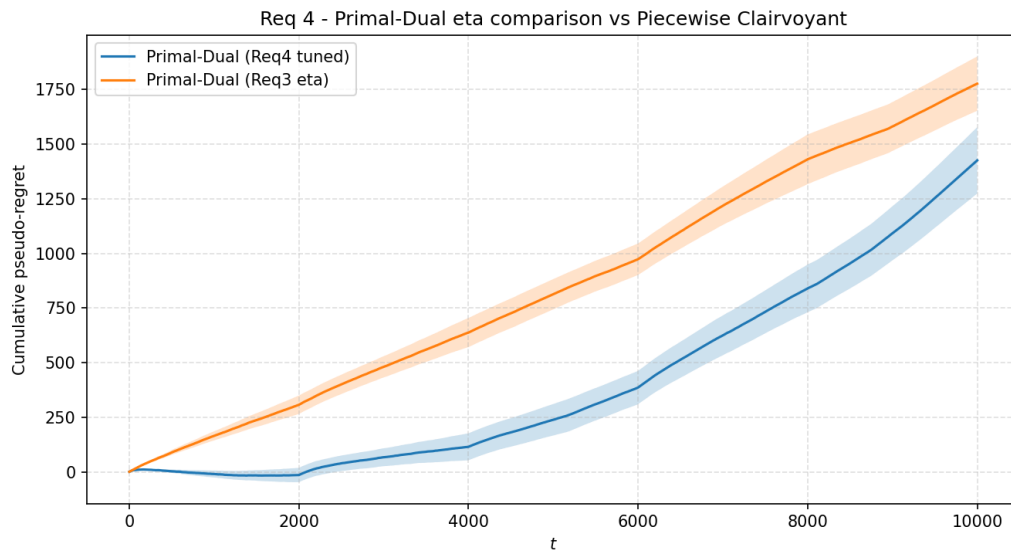


Figure 4: Regret piecewise nel tempo: il retuning riduce sensibilmente il regret lungo tutto l'orizzonte, non solo al termine.

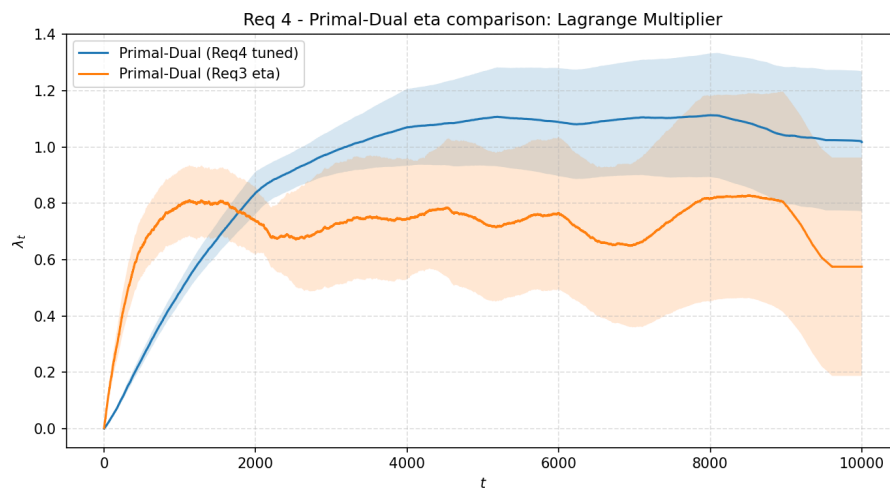


Figure 5: Traiettorie di λ_t : in entrambe le configurazioni λ impiega centinaia di round a risalire dopo ogni shock – il retuning accorcia il ritardo, non lo elimina.